

Série des exercices N°3

Exercice 1 : Le but de ce problème est de gérer les appels(reçus, émis et manqués) des clients d'un opérateur téléphonique. Ces appels sont stockés dans une matrice(M) à n lignes (n est le nombre d'appels enregistrés). Chaque ligne de la matrice contient les informations relatives à un appel donné. Il s'agit du numéro de téléphone(appelé ou appelant)représenté par une chaîne de dix caractères(première colonne de M), la durée d'appel en minutes(deuxième colonne de M) et le type de l'appel(-1 si reçu, 1 si émis et 0 si manqué) (troisième colonne de M) en plus de l'heure(cinq caractères) et la date(dix caractères) stockés respectivement dans la quatrième et la cinquième colonne de M .

Un exemple de la matrice des appels (M) est donné dans le tableau 1 ci-dessous:

Tableau 1

0654342310	5	-1	00h23	31/03/2025
0660324524	0	0	15h10	30/03/2025
0660324524	50	1	15h10	15/03/2025
0660324524	10	1	15h10	22/03/2025
0846394503	60	-1	19h03	01/02/2025
0654394503	13	-1	05h40	04/02/2025

Une matrice en python est, en fait, une liste de listes. Ainsi, la matrice M peut être définie comme suit:

```
M=numpy.array([[['0654342310',5,-1,'10h23','31/05/2012'],
['0660324524',15,0,'11h10','30/05/2012'],['0846394503',60,-1,'09h03','01/06/2012'],
['0654394503',13,1,'09h40','01/06/2012']])
```

Rappel sur les chaînes de caractères:

Les chaînes de caractères sont représentées en Python par le type **str**. Pour affecter la chaîne de caractère 'BONJOUR' à une variable S , il suffit d'écrire $S='BONJOUR'$. Cette chaîne peut, en fait, être considérée comme un tableau de caractères(non modifiable), où chacune de ses cases contient un seul caractère. Ainsi, elle peut être schématisée comme suit:

B	O	N	J	O	U	R
---	---	---	---	---	---	---

Q.1. Vu que certains clients sont dérangés par certains numéros qui les appellent entre minuit (00h00) et 8h(inclues), on envisage définir, pour chaque client, une liste noire qui contiendra tous les numéros qui ont appelé au moins une fois dans cette période. Il est clair

Module: Python pour les sciences de données

qu'un numéro ne doit pas apparaître plus qu'une fois dans la liste noire. Ainsi, il faut avant de l'insérer, vérifier s'il n'y figure pas déjà.

Q.1.a. Ecrire une fonction d'entête *appartient(num,liste)* qui retourne *True* si le numéro *num* figure dans la *liste* et *False* sinon.

Exemple :

`L=['0654342310', '0614352310', '0644342770']`

`Appartient('0654342310', L)` retourne `True`

`Appartient('0611342310', L)` retourne `False`

Q.1.b. Ecrire une fonction d'entête *listeNoire(M)* qui retourne la liste **LNoir** contenant tous les numéros **reçus** entre **minuit et 08h**.

Dans l'exemple du tableau 1ci-dessus, les numéros '0654342310' et '0654394503' devront apparaître dans la liste noire **LNoir**.

Remarques : l'instruction `if ( '00h'<=x<='08h')` permet de vérifier si la chaîne *x* est comprise entre minuit et 08h du matin ou non.

Si *A* est une matrice (deux dimensions) alors `len(A)` retourne le nombre de lignes de *A*.

Q.2) Pour une étude statistique, nous voulons définir pour chaque client les numéros qu'il a appelé ou qui l'ont appelé.

Attention: Un numéro ne doit pas apparaître plus qu'une fois.

Q.2.a) Pour ce faire, on vous demande d'écrire une fonction d'entête *numerosRecus(M)* qui retourne la liste de tous les numéros reçus à l'exception des numéros publicitaires (les numéros publicitaires commencent par '08').

Q.2.b) On vous demande d'écrire une fonction d'entête *numerosAppeles(M)* qui retourne la liste de tous les numéros appelés.

Q.2.b) On vous demande d'écrire une fonction d'entête *numerosManques(M)* qui retourne la liste de tous les numéros manqués à l'exception des numéros publicitaires (les numéros publicitaires commencent par '08').

Q.3. Ecrire une fonction d'entête *DureeTotaleNumeroAppelle(numero, M)* qui retourne la durée totale des appels envoyés vers de ce numéro.

Exemple: Soit la matrice (*M*) des appels ci-dessous:

0654342310	5	-1	17h23	08/03/2025
0654342310	3	1	09h23	27/03/2025
0654342310	9	-1	10h23	04/03/2025
0654342310	0	0	09h23	19/03/2025

Module: Python pour les sciences de données

0654342310	4	1	14h23	17/03/2025
0660324524	0	0	11h10	01/03/2025
0660324524	46	1	15h10	30/03/2025
0660324524	0	0	16h10	25/03/2025
0846394503	60	-1	19h03	22/03/2025
0654394507	13	1	05h40	04/03/2025

L'appel de la fonction *DureeTotaleNumeroAppelle* ('0654342310', *M*) retournera 7=3+4.

L'appel de la fonction *DureeTotaleNumeroAppelle* ('0677445566', *M*) retournera 0.

Q.4. Ecrire une fonction d'entête *NumeroLePlusAppels(M)* qui retourne le numéro le plus appelé, c'est-à-dire celui dont la durée totale des appels envoyés est maximal.

Exemple:

Soit la matrice (*M*) des appels ci-dessous:

0654342310	5	-1	17h23	08/03/2025
0654342310	37	1	09h23	27/03/2025
0654342310	9	-1	10h23	04/03/2025
0654342310	0	0	09h23	19/03/2025
0654342310	41	1	14h23	17/03/2025
0660324524	0	0	11h10	01/03/2025
0660324524	46	1	15h10	30/03/2025
0660324524	0	0	16h10	25/03/2025
0846394503	60	-1	19h03	22/03/2025
0654394507	13	1	05h40	04/03/2025

L'appel de la fonction *NumeroLePlusAppels(M)* retournera le numéro '0654342310'

Exercice 2 : Un carré magique est une matrice de n lignes et n colonnes, dont les coefficients sont les entiers appartenant à $[1, n^2]$ (chaque entier apparaissant une et une seule fois) et telle que les sommes des coefficients de chaque ligne, de chaque colonne et des deux diagonales du carré sont toutes les mêmes.

Exemple :

			= 15
<u>4</u>	9	2	= 15
3	<u>5</u>	7	= 15
8	1	<u>6</u>	= 15
=	=	=	<u>= 15</u>
15	15	15	

Question 1. Programmer une fonction d'entête `sommeline(M, numL)` qui calcule la somme des coefficients de la ligne de numéro `numL`.

Question 2. Programmer une fonction d'entête `sommeacol(M, numC)` qui calcule la somme des coefficients de la colonne de numéro `numC`.

Question 3. Programmer une fonction d'entête `sommediago1(M)` qui calcule la somme des coefficients de la première diagonale

Question 4. Programmer une fonction d'entête `sommediago2(M)` qui calcule la somme des coefficients de la deuxième diagonale.

Question 5. Programmer une fonction `appartenance(M)` qui vérifie si tous les coefficients de la matrice `M` appartiennent à l'intervalle $[1, n^2]$, où n est la dimension de la matrice `M` (retourne `True` si c'est le cas et `False` sinon).

Question 6. Programmer une fonction `repetition(M)` qui permet de vérifier si les coefficients de la matrice `M` ne se répètent pas (retourne `True` si c'est le cas et `False` sinon).

Question 7. Programmer une fonction d'entête `magique(M)` qui vérifie si le carré représenté par la matrice `M` est un carré magique. (retourne `True` si c'est le cas et `False` sinon)

Exercice 3:

Une matrice est dite creuse si le nombre de ses éléments nuls est très important relativement à celui des éléments non nuls (*plus que 50% d'éléments nuls*). La représentation d'une telle matrice par un tableau à 2 dimensions, engendre une occupation importante d'espace mémoire par des 0. Le but de cet exercice est de représenter une telle matrice en optimisant l'espace mémoire utilisé.

Exemple : on considère la matrice suivante :

$$A = \begin{pmatrix} 0 & 2 & 0 & 0 & 0 \\ 1 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 \end{pmatrix}$$

Cette matrice peut être représentée par la liste L suivante :

[2,0,1]	[1,1,0]	[3,1,2]	[6,3,3]
---------	---------	---------	---------

- 1) Ecrire la fonction **defverifMatriceCreuse(M)** qui retourne True si M est creuse, False sinon.
- 2) Ecrire la fonction **def Convertir_Matrice_To_Liste(M)** qui retourne la liste correspondante à la matrice M comme dans l'exemple ci-dessus.
- 3) Ecrire la fonction **def Convertir_Liste_To_Matrice(L,NbrDeLignes,NbrDeColonnes)** qui retourne la matrice M correspondante à la liste L donnée en argument.